

AI Systems Foundations

How real AI products are architected — beyond a single API call

What This Module Is About

Most developers learn to call an LLM API and get a response. Real AI products are far more complex — they are orchestrated systems of models, memory, tools, infrastructure, and reliability engineering. This module teaches you to think in systems, not scripts.

The Three Roles in an AI Team

Role	Focus	You Are Here
AI Engineer	APIs, agents, orchestration, system design	YES — this course
ML Engineer	Loss functions, training, evaluation metrics	Related, not covered here
Data Scientist	Dashboards, reporting, data pipelines	Related, not covered here

Key Concepts

- **Orchestrated Systems** — a production AI product combines models, memory, tools, and infrastructure. No single component works in isolation.
- **Memory Management** — AI systems need working memory (current task), episodic memory (session history), and sometimes semantic memory (long-term knowledge).
- **Async Task Queues** — long-running AI tasks (embeddings, generation) should run asynchronously via queues like Celery or RQ, not block HTTP responses.
- **Latency & Observability** — track token usage, response times, and error rates from day one. You cannot improve what you cannot measure.
- **Separation of Concerns** — never hardcode AI calls into UI logic or business logic. Each layer has one job.

Tip: Draw your system architecture BEFORE you write any code. A simple box-and-arrow diagram of components and data flow will save you hours of refactoring later.

Antipatterns to Avoid

- ✗ Hardcoding model API calls directly inside UI handlers or route functions.
- ✗ No .env file — API keys committed to Git (a security disaster).

- ✗ Building locally with no plan for containers, CI/CD, or cloud deployment.
- ✗ Missing latency monitoring — you have no idea if your system is slow.
- ✗ No error handling — one failed API call crashes your entire app.

Resources for Deeper Learning

1

System Design Primer (GitHub)

<https://github.com/donnmartin/system-design-primer>

The most-starred system design resource on GitHub. Study the sections on scalability, databases, caching, and async processing — all apply directly to AI systems.

2

FastAPI — Background Tasks & Async

<https://fastapi.tiangolo.com/tutorial/background-tasks/>

Official FastAPI docs on running background tasks and async endpoints. Essential for handling slow AI operations without blocking your API.

3

Celery — Distributed Task Queue

<https://docs.celeryq.dev/en/stable/getting-started/introduction.html>

Celery is the standard Python tool for async task queues. Use it when your AI tasks take more than a few seconds to complete.

4

OpenTelemetry for Python

<https://opentelemetry.io/docs/languages/python/>

Industry-standard observability framework. Add tracing and metrics to your AI system from the start so you can debug latency issues in production.

5

Twelve-Factor App Methodology

<https://12factor.net/>

A short, essential read on how to build production-ready applications. Principles like config in environment variables, stateless processes, and log streaming apply directly.

6

LangChain Architecture Concepts

<https://python.langchain.com/docs/concepts/>

Even if you don't use LangChain, reading its architecture concepts (chains, memory, tools) will deepen your mental model of AI system design.

Module 2 Assignment — Design Your AI System Architecture

- Draw a box-and-arrow diagram of a simple AI assistant: User → API → LLM → Response. Label each component with its responsibility.
- Extend the diagram to include: memory store, tool integrations, async task queue, and error handling layer. Add this diagram to your GitHub README.
- Write a FastAPI endpoint that calls the Claude API with a user message and streams back the response. Use .env for the API key.
- Add basic logging to your endpoint: log the request timestamp, token count, and latency (in milliseconds) for each API call.
- Identify ONE antipattern from the list above that you might have written naturally — refactor your code to fix it.

Note: Your architecture diagram will become the blueprint for your capstone project.